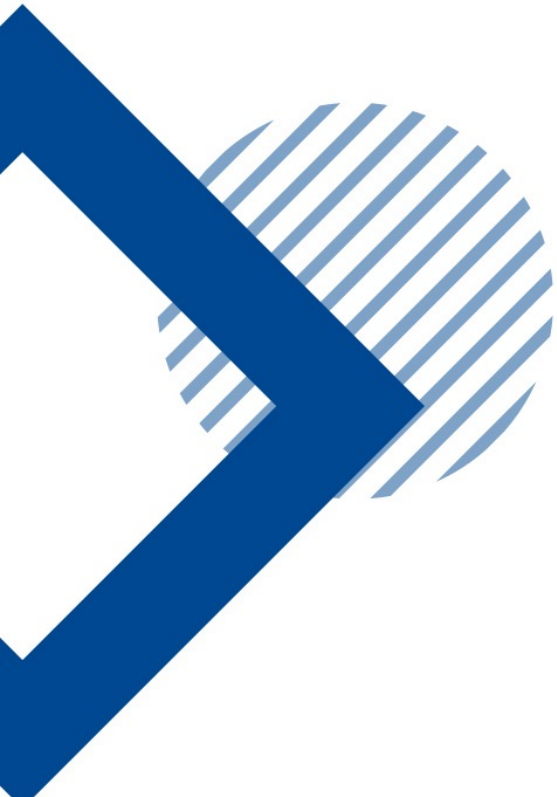
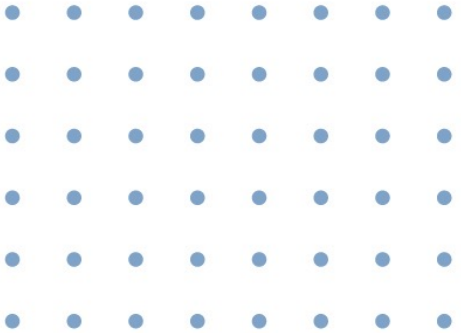
Vue.js

Интернет-программирование







2022

Vue.js

Vue.js (также Vue) — JavaScript-фреймворк с открытым исходным кодом для создания клиентских приложений и интерфейсов.

Vue.js легко интегрируется в проекты с использованием других JavaScript-библиотек. Может функционировать как веб-фреймворк для разработки одностраничных приложений в реактивном стиле.

Реактивное программирование — парадигма программирования, ориентированная на потоки данных и распространение изменений.

Современные табличные процессоры представляют собой пример реактивного программирования. Ячейки таблицы могут содержать строковые значения или формулу вида «=B1+C1», значение которой будет вычислено исходя из значений соответствующих ячеек. Когда значение одной из зависимых ячеек будет изменено, значение этой ячейки будет автоматически обновлено.

Шаблонизатор

Вывод текста:
*Сообщение: **{{ msg }}***

Задание значения атрибута:
*<div **v-bind:id**="**dynamicId**"> *
*<div **:id**="**dynamicId**"> *

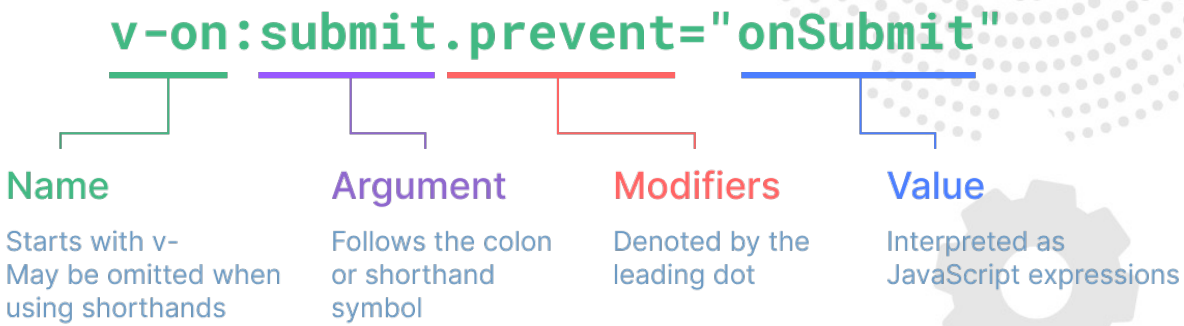
*<div **:id**="**'list-' + id**"> *
*<div **:id**="`**list-${id}**`"> *

Аргументы директив:
*<a **:href**="url"> ... *
*<a **v-on:click**="doSomething"> ... *
*<a **@click**="doSomething(arg)"> ... *

Модификаторы:
*<a **@click.stop.prevent**="doSomething(arg)"> ... *

Динамические аргументы:
*<a **:[attributeName]**="url"> ... *
*<a **@[eventName]**="doSomething"> ... *

*<p **v-if="seen"**>Сейчас меня видно</p>*



Реактивность

```
import { ref } from 'vue'

const count = ref(0)

console.log(count) // { value: 0 }
console.log(count.value) // 0

count.value++
console.log(count.value) // 1

function increment() {
  count.value++
}
```

```
import { reactive } from 'vue'

const state = reactive({ count: 0 })
```

Вычисляемые свойства

```
const author = reactive({
  name: 'John Doe',
  books: [
    'Vue 2 - Advanced Guide',
    'Vue 3 - Basic Guide',
    'Vue 4 - The Mystery'
  ]
})

const publishedBooksMessage = computed(() => {
  return author.books.length > 0 ? 'Yes' : 'No’
})

<p>Has published books:</p>
<span>{{ publishedBooksMessage }}</span>
```

Вычисляемые свойства с сеттером

```
import { ref, computed } from 'vue'

const firstName = ref('John')
const lastName = ref('Doe')

const fullName = computed({
  // getter
  get() {
    return firstName.value + ' ' + lastName.value
  },
  // setter
  set(newValue) {.
    [firstName.value, lastName.value] = newValue.split(' ')
  }
})
```

Работа со стилями 1

```
const isActive = ref(true)
const hasError = ref(false)

<div
  class="static"
  :class="{ active: isActive, 'text-danger': hasError }"
></div>


const classObject = reactive({
  active: true,
  'text-danger': false
})

<div :class="classObject"></div>

const isActive = ref(true)
const error = ref(null)

const classObject = computed(() => ({
  active: isActive.value && !hasError.value,
  'text-danger': hasError.value
}))
```

Работа со стилями 2

```
const activeClass = ref('active')
const errorClass = ref('text-danger’)

<div :class="[activeClass, errorClass]"></div>

<div :class="[isActive ? activeClass : '', errorClass]"></div>

<div :class="[{ active: isActive }, errorClass]"></div>


MyComponent
<p class="foo bar">Hi!</p>

<MyComponent class="baz boo" />

<p class="foo bar baz boo">Hi!</p>


<MyComponent :class="{ active: isActive }" />
```

Условный рендеринг 1

```
<h1 v-if="awesome">Vue is awesome!</h1>


<button @click="awesome = !awesome">Toggle</button>

<h1 v-if="awesome">Vue is awesome!</h1>
<h1 v-else>Oh no 😢</h1>


<div v-if="type === 'A'">
  A
</div>
<div v-else-if="type === 'B'">
  B
</div>
<div v-else-if="type === 'C'">
  C
</div>
<div v-else>
  Not A/B/C
</div>
```

Условный рендеринг 2

```
<template v-if="ok">
  <h1>Title</h1>
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</template>
```

```
<h1 v-show="ok">Hello!</h1>
```

Рендеринг списков 1

```
const items = ref([{ message: 'Foo' }, { message: 'Bar' }])

<li v-for="item in items">
  {{ item.message }}
</li>

<li v-for="(item, index) in items">
  {{ index }} - {{ item.message }}
</li>

<li v-for="({ message }, index) in items">
  {{ message }} {{ index }}
</li>


<li v-for="item in items">
  <span v-for="childItem in item.children">
    {{ item.message }} {{ childItem }}
  </span>
</li>
```

Рендеринг списков 2

```
<span v-for="n in 10">{{ n }}</span>
```

```
<ul>
  <template v-for="item in items">
    <li>{{ item.msg }}</li>
    <li class="divider" role="presentation"></li>
  </template>
</ul>
```

```
<template v-for="todo in todos" :key="todo.name">
  <li>{{ todo.name }}</li>
</template>
```

```
<MyComponent v-for="item in items" :key="item.id" />
```

Использовать v-for с v-if не рекомендуется.

Используйте вычисляемые свойства, в которых выполняется фильтрация коллекции.

Обработка событий

```
<a @click.stop="doThis"></a>

<form @submit.prevent="onSubmit"></form>

<a @click.stop.prevent="doThat"></a>

<form @submit.prevent></form>

<div @click.self="doThat">...</div>


<input @keyup.enter="submit" />

<input @keyup.page-down="onPageDown" />

<input @keyup.alt.enter="clear" />
```

Работа с формой

```
<input
  :value="text"
  @input="event => text = event.target.value">
```

```
<input v-model="text">
```

<input>, <textarea> – :value и @input

<input type="checkbox">, <input type="radio"> – :checked и @change

<select> – :value и @change

Watcher 1

```
import { ref, watch } from 'vue'

const question = ref('')
const answer = ref('Questions usually contain a question mark. ;-)')
const loading = ref(false)

watch(question, async (newQuestion, oldQuestion) => {
  if (newQuestion.includes('?')) {
    loading.value = true
    answer.value = 'Thinking...'
    try {
      const res = await fetch('https://yesno.wtf/api')
      answer.value = (await res.json()).answer
    } catch (error) {
      answer.value = 'Error! Could not reach the API. ' + error
    } finally {
      loading.value = false
    }
  }
})
```

Watcher 2

```
const x = ref(0)
const y = ref(0)

// single ref
watch(x, (newX) => {
  console.log(`x is ${newX}`)
})

// getter
watch(
  () => x.value + y.value,
  (sum) => {
    console.log(`sum of x + y is: ${sum}`)
  }
)

// array of multiple sources
watch([x, () => y.value], ([newX, newY]) => {
  console.log(`x is ${newX} and y is ${newY}`)
})
```

Watcher 3

```
watch(
  source,
  (newValue, oldValue) => {
    // executed immediately, then again when `source` changes
  },
  { immediate: true }
)


const unwatch = watchEffect(() => {})

unwatch()


watchEffect(async () => {
  const response = await fetch(
    `https://jsonplaceholder.typicode.com/todos/${todoId.value}`
  )
  data.value = await response.json()
})
```

Ссылки на элементы DOM

```
import { ref, onMounted } from 'vue'

const input = ref(null)

onMounted(() => {
  input.value.focus()
})
```

```
<input ref="input" />
```

Компонент

```
<script setup>
import { ref } from 'vue'

const count = ref(0)
</script>
<template>
  <button class=“my-class” @click="count++">You clicked me {{ count }} times.</button>
</template>
<style scoped>
  .my-class: {
    color: red;
  }
</style>

<script setup>
import ButtonCounter from './ButtonCounter.vue'
</script>
<template>
  <h1>Here is a child component!</h1>
  <ButtonCounter />
  <ButtonCounter />
</template>
```

Компонент Vue 2

```
<script>
   export default {
      data() {
         return { count: 0 }
      },
      methods: {
         increment() {
            this.count++;
         }
      }
   }
</script>

<template>
   <p>Значение счетчика {{ count }}</p>
   <button @click="increment">Увеличить счетчик</button>
</template>
```

Свойства компонента

```
defineProps({
  title: String,
  likes: Number
})
```

```
<span>{{ title }}</span>
<span>{{ likes }}</span>
```

```
<MyComponent title=”Example” :likes=“likes” />
```

Валидация свойств компонента

```
definePropsA({
  propA: Number,
  propB: [String, Number],
  propC: {
    type: String,
    required: true
  },
  propD: {
    type: Number,
    default: 100
  },
  propE: {
    type: Object,
    default(rawProps) {
      return { message: 'hello' }
    }
  },
  propF: {
    validator(value) {
     return ['success', 'warning', 'danger'].includes(value)
    }
  },
  propG: {
    type: Function,
    default() {
      return 'Default function'
    }
  }
})
```

События компонента

```
defineEmits(['increaseBy '])

function buttonClick() {
  emit('increaseBy', 1)
}

<button @click=" buttonClick ">Increase by 1</button>


<button @click="$emit('increaseBy', 1)">Increase by 1</button>




<MyButton @increase-by="(n) => count += n" />


<MyButton @increase-by="increaseCount" />

function increaseCount(n) {
  count.value += n
}
```

Валидация событий

```
const emit = defineEmits({
  click: null,

  submit: ({ email, password }) => {
    if (email && password) {
      return true
    } else {
      console.warn('Invalid submit event payload!')
      return false
    }
  }
})

function submitForm(email, password) {
  emit('submit', { email, password })
}
```

v-model для компонента 1

```
<script setup>
defineProps(['modelValue'])
defineEmits(['update:modelValue'])
</script>

<template>
  <input
    :value="modelValue"
    @input="$emit('update:modelValue', $event.target.value)"
  />
</template>


<CustomInput v-model="searchText" />
```

v-model для компонента 2

```
<script setup>
import { computed } from 'vue'

const props = defineProps(['modelValue'])
const emit = defineEmits(['update:modelValue'])

const value = computed({
  get() {
    return props.modelValue
  },
  set(value) {
    emit('update:modelValue', value)
  }
})
</script>

<template>
  <input v-model="value" />
</template>
```

v-model для компонента 3

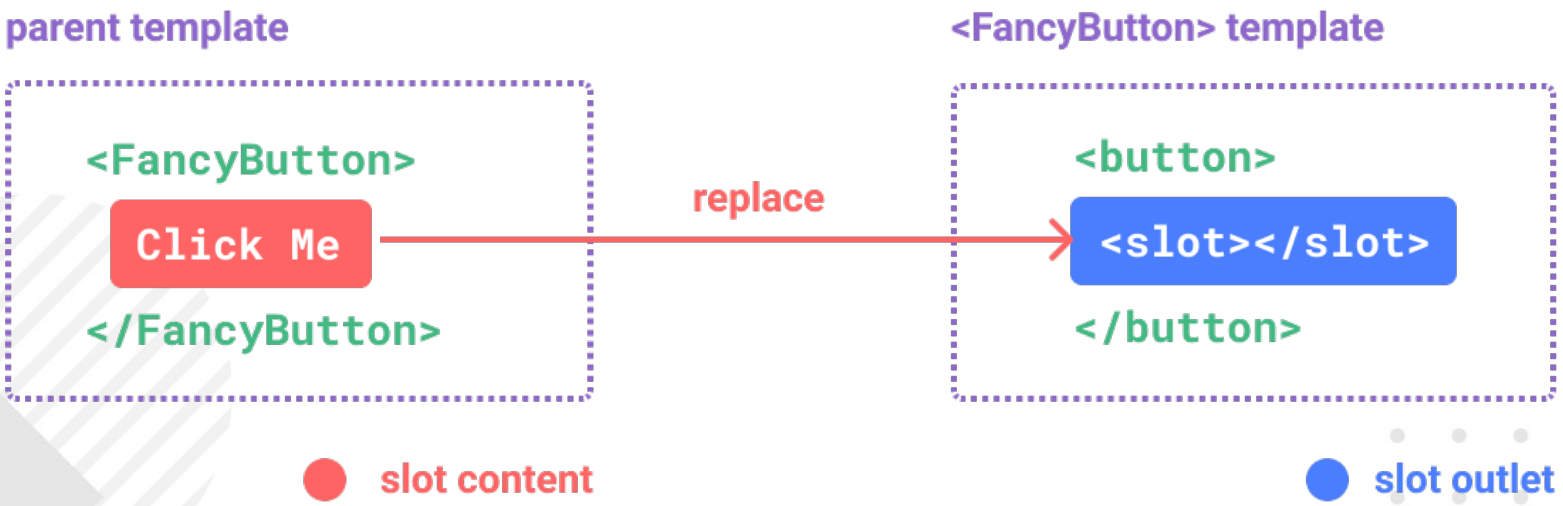
```
<script setup>
defineProps(['title'])
defineEmits(['update:title'])
</script>

<template>
  <input
    type="text"
    :value="title"
    @input="$emit('update:title', $event.target.value)"
  />
</template>
```

Слоты 1

```
<FancyButton>
  Click me! <!-- slot content -->
</FancyButton>
```

```
<button class="fancy-btn">
  <slot></slot> <!-- slot outlet -->
</button>
```

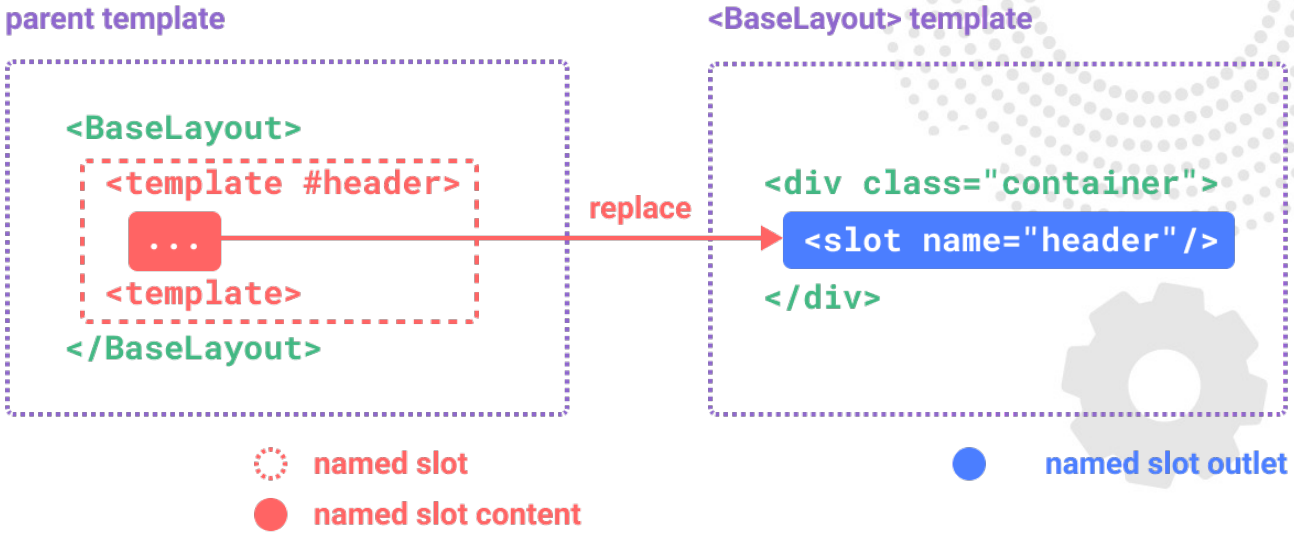


Слоты 2

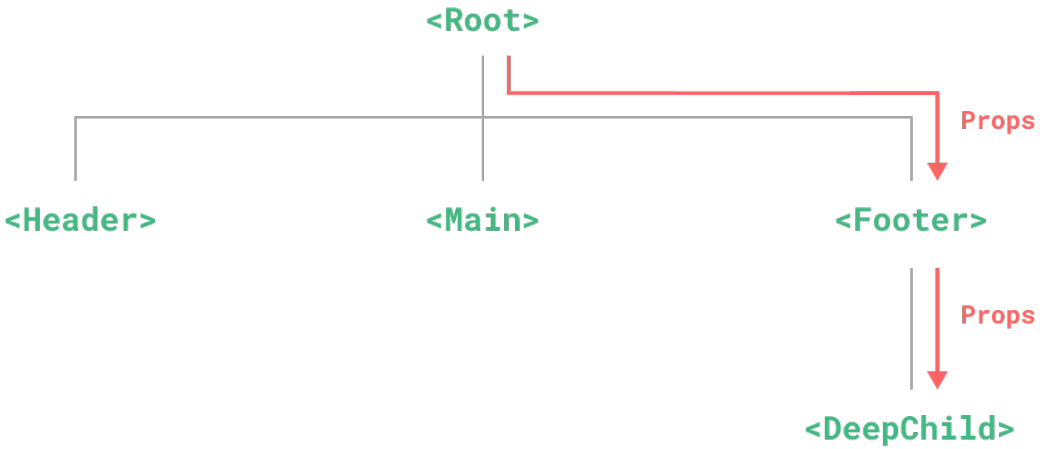
```
<button type="submit">
  <slot>
    Submit <!-- fallback content -->
  </slot>
</button>
```

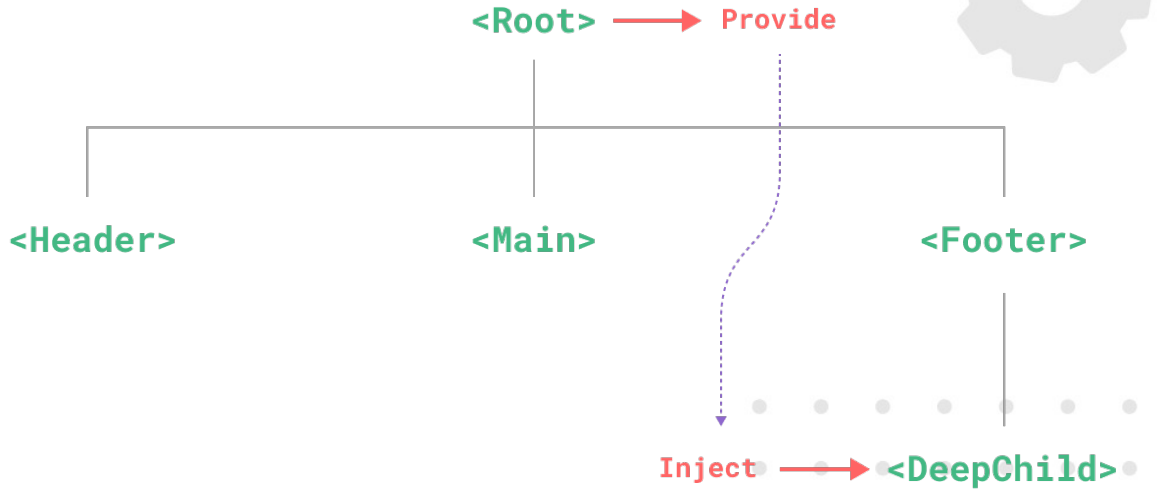
```
<div class="container">
  <header>
    <slot name="header"></slot>
  </header>
  <main>
    <slot></slot>
  </main>
  <footer>
    <slot name="footer"></slot>
  </footer>
</div>
```

```
<BaseLayout>
  <template #header>
    <!-- content for the header slot -->
  </template>
</BaseLayout>
```



Provide / Inject 1





Provide / Inject 2

```
<script setup>
import { ref, provide } from 'vue'

const count = ref(0)
provide('key', count)
provide('read-only-count', readonly(count))
</script>
```

```
<script setup>
import { inject } from 'vue'

const count = inject(‘count’, 10)
</script>
```

```
<script setup>
import { provide, ref } from 'vue'

const location = ref('North Pole')

function updateLocation() {
  location.value = 'South Pole'
}

provide('location', {
  location,
  updateLocation
})
</script>
```

Композиция

```
import { ref, onMounted, onUnmounted } from 'vue'

export function useMouse() {
  const x = ref(0)
  const y = ref(0)

  function update(event) {
    x.value = event.pageX
    y.value = event.pageY
  }

  onMounted(() =>
    window.addEventListener('mousemove', update))
  onUnmounted(() =>
     window.removeEventListener('mousemove', update))

  return { x, y }
}
```

```
<script setup>
import { useMouse } from './mouse.js'

const { x, y } = useMouse()
</script>

<template>
  Mouse position is at: {{ x }}, {{ y }}
</template>
```

Спасибо за внимание!

Литература:

1. Документация Vue - https://vuejs.org/guide/introduction.html
2. События жизненного цикла - https://vuejs.org/guide/essentials/lifecycle.html
3. Основы компонентов - https://vuejs.org/guide/essentials/component-basics.html
4. Композиция - https://vuejs.org/guide/reusability/composables.html
5. Управление состоянием - https://vuejs.org/guide/scaling-up/state-management.html
6. Работа с формами - https://vuejs.org/guide/essentials/forms.html
7. Реактивность - https://vuejs.org/api/reactivity-core.html#computed
8. Маршрутизация - https://router.vuejs.org/guide/



2022